

PRAKTIKUM BASIS DATA

MODUL 13

(NoSQL - Document Database)



Oleh:

Danu Warisman

103122400041

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

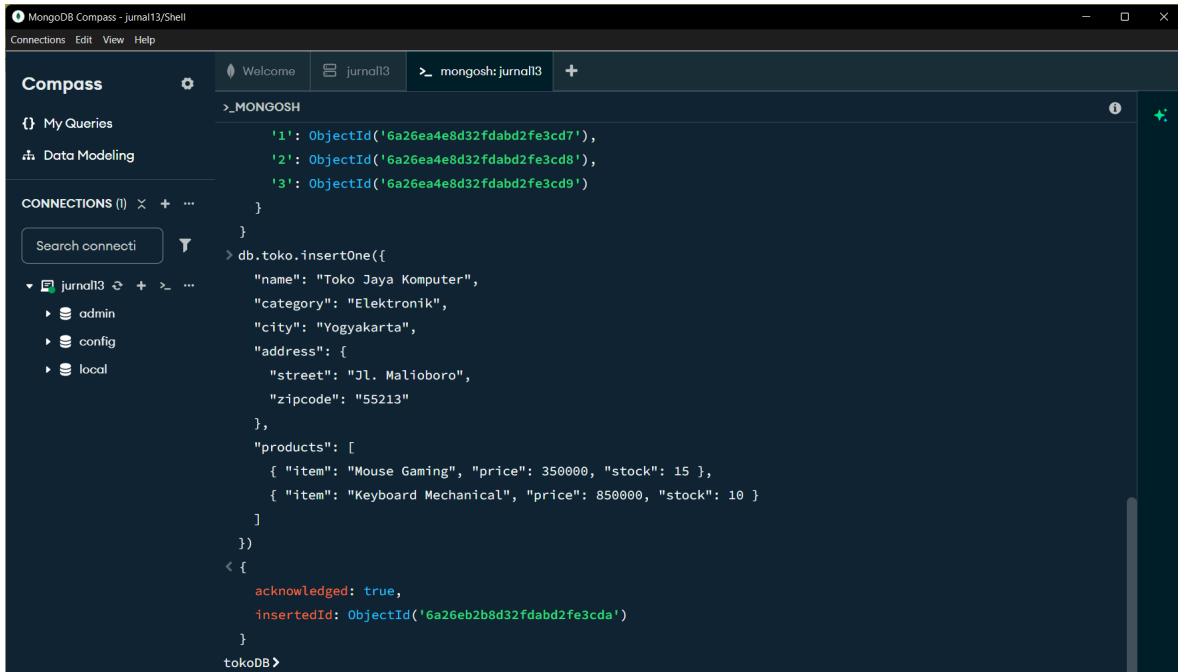
DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

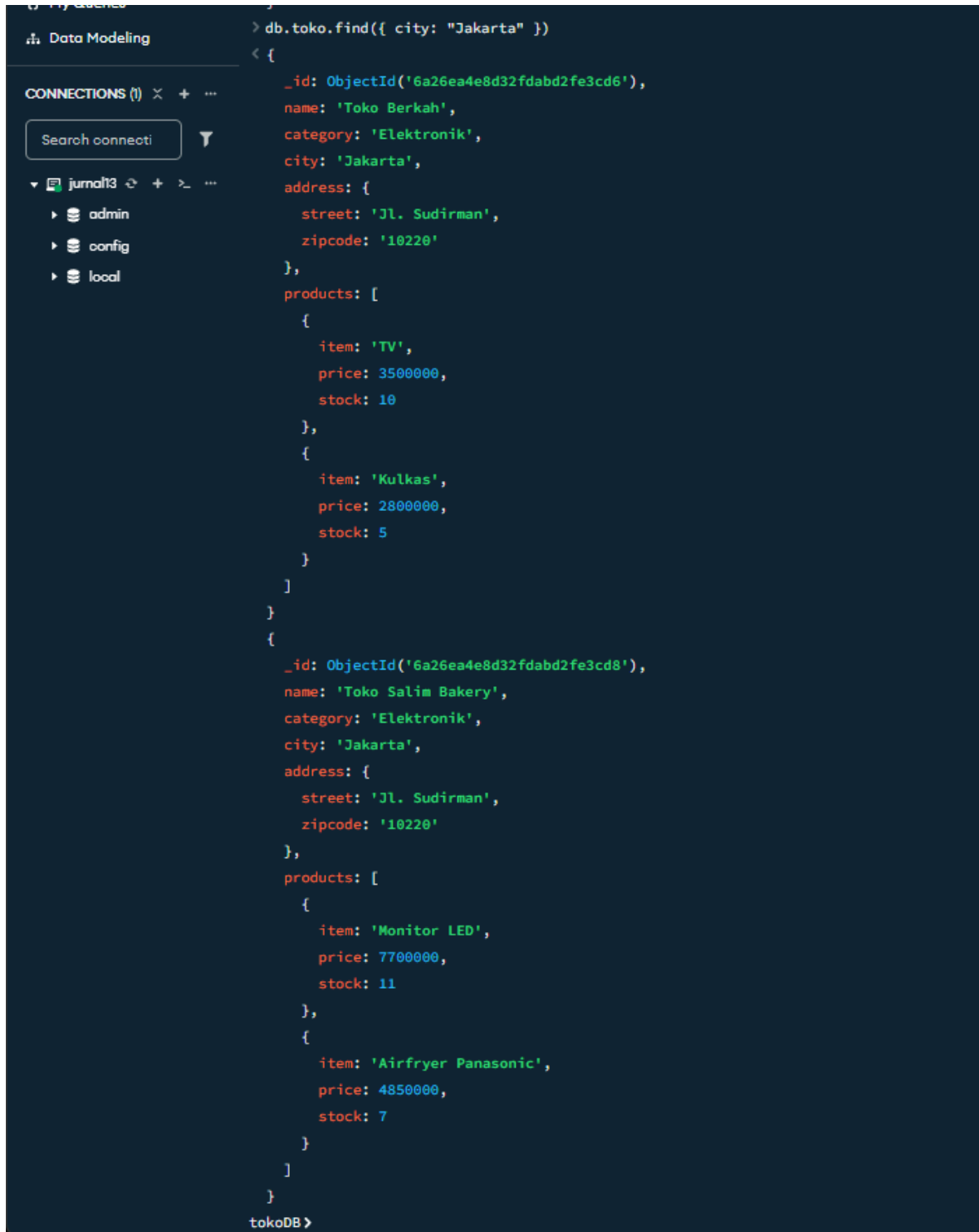
JURNAL

1. menambahkan 1 toko sesuai ketentuan



2. menampilkan semua toko yang ada di jakarta

db.toko.find({ city: "Jakarta" })



The screenshot shows a MongoDB shell interface with a dark theme. On the left, there is a sidebar with a 'Data Modeling' icon and a 'CONNECTIONS (1)' section. Below this, there is a search bar labeled 'Search connections' and a list of connections: 'jurnal13', 'admin', 'config', and 'local'. The main area of the shell displays the command `> db.toko.find({ city: "Jakarta" })` and its output. The output is a JSON array containing two shop objects. Each object has fields for `_id`, `name`, `category`, `city`, `address` (with `street` and `zipcode` sub-fields), and `products` (an array of product objects with `item`, `price`, and `stock` fields).

```
> db.toko.find({ city: "Jakarta" })
< {
  _id: ObjectId('6a26ea4e8d32fdabd2fe3cd6'),
  name: 'Toko Berkah',
  category: 'Elektronik',
  city: 'Jakarta',
  address: {
    street: 'Jl. Sudirman',
    zipcode: '10220'
  },
  products: [
    {
      item: 'TV',
      price: 3500000,
      stock: 10
    },
    {
      item: 'Kulkas',
      price: 2800000,
      stock: 5
    }
  ]
},
{
  _id: ObjectId('6a26ea4e8d32fdabd2fe3cd8'),
  name: 'Toko Salim Bakery',
  category: 'Elektronik',
  city: 'Jakarta',
  address: {
    street: 'Jl. Sudirman',
    zipcode: '10220'
  },
  products: [
    {
      item: 'Monitor LED',
      price: 7700000,
      stock: 11
    },
    {
      item: 'Airfryer Panasonic',
      price: 4850000,
      stock: 7
    }
  ]
}
]
}
tokodb >
```

3. menampilkan toko yang memiliki produk dengan price lebih dari 3.000.000.

```
db.toko.find({ "products.price": { $gt: 3000000 } })
```

The image displays two side-by-side screenshots of the MongoDB Compass interface, showing the results of a query executed in the Shell. The query is `db.toko.find({ "products.price": { $gt: 3000000 } })`.

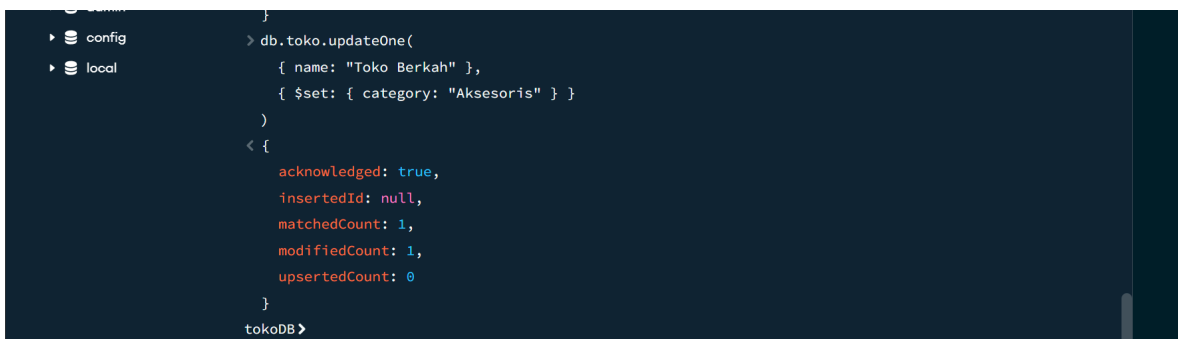
Left Screenshot: The query results show two shops. The first shop, 'Toko Berkah', is located in Jakarta and has two products: 'TV' (price 3,500,000, stock 10) and 'Kulkas' (price 2,800,000, stock 5). The second shop, 'Toko Maju', is located in Bandung and has two products: 'HP' (price 1,500,000, stock 20) and 'Laptop' (price 8,000,000, stock 3).

Right Screenshot: The query results show two shops. The first shop, 'Toko Salim Bakery', is located in Jakarta and has two products: 'Monitor LED' (price 7,700,000, stock 11) and 'Airfryer Panasonic' (price 4,850,000, stock 7). The second shop, 'Toko Sudinotech', is located in Surabaya and has two products: 'RTX 55000' (price 8,500,000, stock 20) and 'Set PC Gaming Radeon' (price 12,000,000, stock 9).

4. mengubah category Toko Berkah dari Fashion menjadi Aksesoris Dan menambahkan 1 produk baru ke dalam array products milik Toko Maju menggunakan \$push.

a. mengubah kategori

```
db.toko.updateOne(  
  { name: "Toko Berkah" },  
  { $set: { category: "Aksesoris" } }  
)
```



```
config  
local  
> db.toko.updateOne(  
  { name: "Toko Berkah" },  
  { $set: { category: "Aksesoris" } }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
tokoDB>
```

b. menambahkan satu produk

```
db.toko.updateOne(  
  { name: "Toko Maju" },  
  { $push: { products: { item: "Mesin fotokopi", price: 2500000, stock: 8 } } }  
)
```



```
config  
local  
> db.toko.updateOne(  
  { name: "Toko Maju" },  
  { $push: { products: { item: "Mesin fotokopi", price: 2500000, stock: 8 } } }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
tokoDB>
```

5. menghitung jumlah toko per city, urutkan dari kota yang paling banyak tokonya

```
db.toko.aggregate([
  { $group: { _id: "$city", jumlah_toko: { $sum: 1 } } },
  { $sort: { jumlah_toko: -1 } }
])
```



The screenshot shows a MongoDB query interface with a sidebar on the left containing 'My Queries', 'Data Modeling', and 'CONNECTIONS'. The main editor displays the following query:

```
> db.toko.aggregate([
  { $group: { _id: "$city", jumlah_toko: { $sum: 1 } } },
  { $sort: { jumlah_toko: -1 } }
])
```

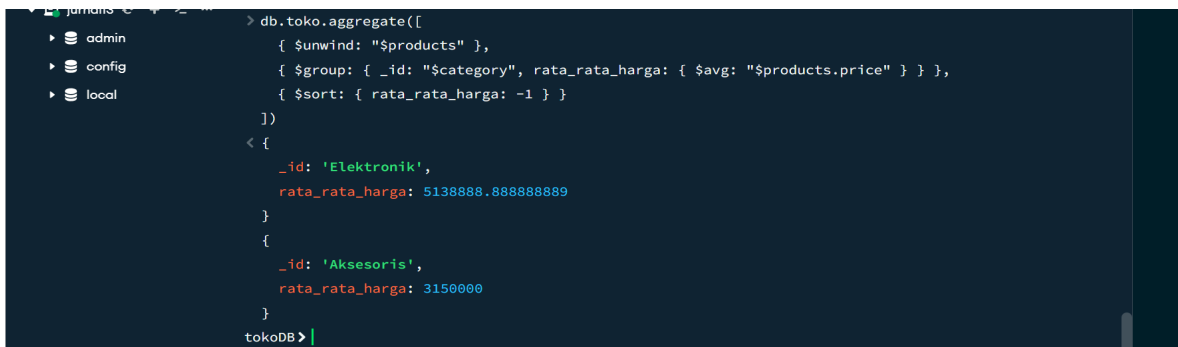
The result is displayed as a JSON array:

```
< [
  {
    "_id": "Jakarta",
    "jumlah_toko": 2
  },
  {
    "_id": "Yogyakarta",
    "jumlah_toko": 1
  },
  {
    "_id": "Surabaya",
    "jumlah_toko": 1
  },
  {
    "_id": "Bandung",
    "jumlah_toko": 1
  }
]
```

The prompt 'tokodb >' is visible at the bottom of the editor.

6. menghitung rata-rata harga produk per category

```
db.toko.aggregate([
  { $unwind: "$products" },
  { $group: { _id: "$category", rata_rata_harga: { $avg: "$products.price" } } },
  { $sort: { rata_rata_harga: -1 } }
])
```



The screenshot shows a MongoDB query interface with a sidebar on the left containing 'My Queries', 'Data Modeling', and 'CONNECTIONS'. The main editor displays the following query:

```
> db.toko.aggregate([
  { $unwind: "$products" },
  { $group: { _id: "$category", rata_rata_harga: { $avg: "$products.price" } } },
  { $sort: { rata_rata_harga: -1 } }
])
```

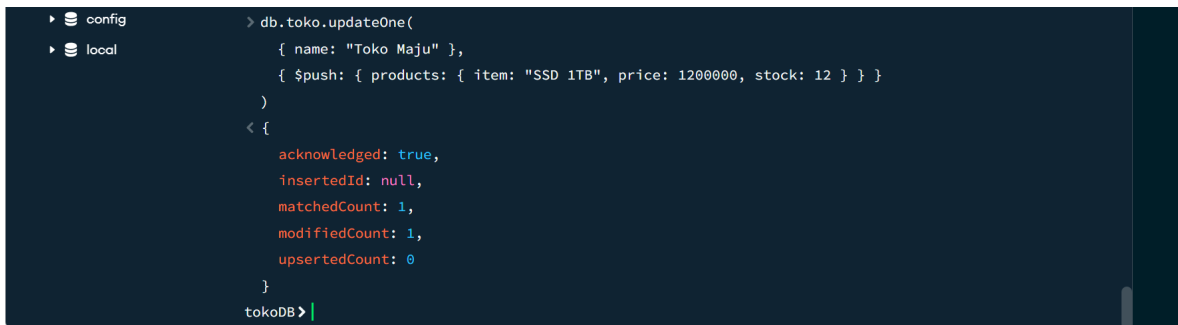
The result is displayed as a JSON array:

```
< [
  {
    "_id": "Elektronik",
    "rata_rata_harga": 5138888.888888889
  },
  {
    "_id": "Aksesoris",
    "rata_rata_harga": 3150000
  }
]
```

The prompt 'tokodb >' is visible at the bottom of the editor.

7. menambahkan produk baru ke dalam array products milik Toko Maju menggunakan \$push

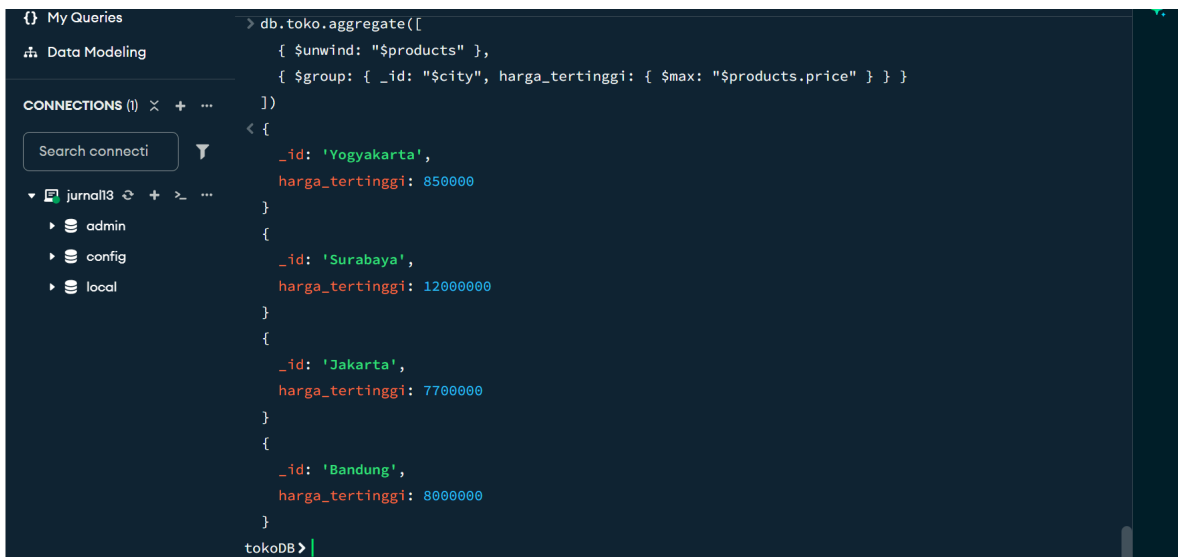
```
db.toko.updateOne(  
  { name: "Toko Maju" },  
  { $push: { products: { item: "SSD 1TB", price: 1200000, stock: 12 } } }  
)
```



```
> db.toko.updateOne(  
  { name: "Toko Maju" },  
  { $push: { products: { item: "SSD 1TB", price: 1200000, stock: 12 } } }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

8. menampilkan harga produk tertinggi (\$max) per city

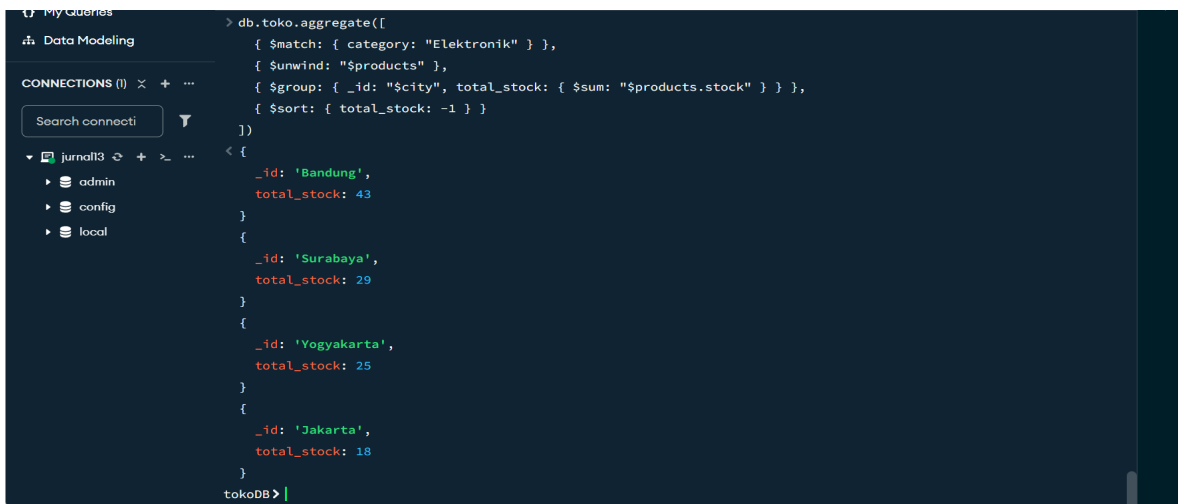
```
db.toko.aggregate([  
  { $unwind: "$products" },  
  { $group: { _id: "$city", harga_tertinggi: { $max: "$products.price" } } }  
)
```



```
> db.toko.aggregate([  
  { $unwind: "$products" },  
  { $group: { _id: "$city", harga_tertinggi: { $max: "$products.price" } } }  
)  
< {  
  _id: 'Yogyakarta',  
  harga_tertinggi: 850000  
}  
{  
  _id: 'Surabaya',  
  harga_tertinggi: 1200000  
}  
{  
  _id: 'Jakarta',  
  harga_tertinggi: 770000  
}  
{  
  _id: 'Bandung',  
  harga_tertinggi: 800000  
}
```

9. Buat pipeline lengkap

```
db.toko.aggregate([
  { $match: { category: "Elektronik" } },
  { $unwind: "$products" },
  { $group: { _id: "$city", total_stock: { $sum: "$products.stock" } } },
  { $sort: { total_stock: -1 } }
])
```



```
> db.toko.aggregate([
  { $match: { category: "Elektronik" } },
  { $unwind: "$products" },
  { $group: { _id: "$city", total_stock: { $sum: "$products.stock" } } },
  { $sort: { total_stock: -1 } }
])
< {
  _id: 'Bandung',
  total_stock: 43
}
{
  _id: 'Surabaya',
  total_stock: 29
}
{
  _id: 'Yogyakarta',
  total_stock: 25
}
{
  _id: 'Jakarta',
  total_stock: 18
}
tokoDB>
```

10. Hapus toko yang memenuhi dua kondisi sekaligus

```
db.toko.deleteOne({
  $and: [
    { category: "Fashion" },
    { city: "Surabaya" }
  ]
})
```



```
> db.toko.deleteOne({
  $and: [
    { category: "Fashion" },
    { city: "Surabaya" }
  ]
})
< {
  acknowledged: true,
  deletedCount: 0
}
tokoDB>
```